ACADEMIC CAPSTONE / DEGREE PROJECT REPORT

IT Vendor Renewal Decision Support Agent

An Evidence-Based Decision Support System for IT Software Contract Renewals, Licence Utilisation, SLA Compliance, and Cost Governance

Candidate Name:	[Student Name: Sagar Vishwakarma]
Student / Roll ID:	[Student ID / Enrollment Number]
Degree / Course:	[Degree / Course Name, e.g., B.Tech in Computer Science / MCA / B.Sc. IT]
Academic Department:	[Department of Computer Science and Engineering]
Institution / University:	[Institution / University Name, City, State]
Project Supervisor / Guide:	[Supervisor / Faculty Guide Name, Designation]
Submission Date:	[Submission Date: October 2026]
Project Operational Mode:	**Strictly Advisory (Human-in-the-Loop Governance)**

Submitted in partial fulfilment of the academic requirements for the award of the designated degree.

Certificate of Approval

This is to certify that the project entitled **"IT Vendor Renewal Decision Support Agent"** submitted by `[Student Name]` (`[Student ID / Enrollment Number]`) in partial fulfilment of the requirements for the degree of `[Course / Degree Name]` at `[Institution / University Name]` is an authentic record of bonafide work carried out under my supervision and guidance during the academic session `[Academic Year, e.g., 2025–2026]`.

The project implements an automated, deterministic advisory system for evaluating IT software vendor contracts, licence allocation, service level agreements (SLAs), and renewal quotation costs without autonomously modifying or executing contractual agreements. The matter presented in this report has not been submitted elsewhere for the award of any other degree or diploma.

`[Supervisor / Guide Name]`
Internal Project Guide
Department of `[Department Name]`
`[Institution Name]`
	`[Head of Department Name]`
	Head of Department
	Department of `[Department Name]`
	`[Institution Name]`

Internal Examiner Date: `[DD/MM/YYYY]`	**External Examiner** Date: `[DD/MM/YYYY]`

Candidate Declaration

I, `[Student Name]`, student of `[Course / Degree Name]`, bearing Enrollment Number `[Student ID]`, hereby declare that the project report entitled **"IT Vendor Renewal Decision Support Agent"** is an original and authentic piece of software engineering and research work carried out by me under the supervision of `[Supervisor Name]`.

I solemnly affirm that:

1. All system architectures, analytical algorithms, data processing workflows, and verification test suites detailed herein were constructed and tested as described.
2. Where existing libraries, parsing utilities, or reference frameworks have been utilized, they have been explicitly acknowledged and cited.
3. The software has been intentionally architected as an **advisory-only decision support tool**, explicitly disallowing automated contract execution, termination, signing, or unilateral price modification.

4. This report has not been previously submitted in part or full to any university, examination board, or institution for the conferral of any academic degree, diploma, or certificate.

`[Signature of the Student]`
`[Student Name]`
Student Roll Number: `[Student ID]`
Date: `[Submission Date]`

Acknowledgements

The successful conceptualisation, architectural design, implementation, and rigorous verification of the **IT Vendor Renewal Decision Support Agent** has been a profoundly enriching academic journey, made possible through the guidance, encouragement, and technical assistance of numerous individuals and institutional mentors.

First and foremost, I express my deepest gratitude to my esteemed project supervisor, `[Supervisor / Guide Name]`, `[Supervisor Designation]`, Department of `[Department Name]`, for invaluable guidance, patience, and insightful critiques throughout the project lifecycle. Their insistence on deterministic financial accuracy, rigorous edge-case verification, and ethical governance standards profoundly shaped the system's architecture.

I extend my sincere appreciation to `[Head of Department Name]`, Head of the Department, and the distinguished faculty members of `[Institution Name]` for fostering a stimulating academic environment and providing state-of-the-art laboratory and computational infrastructure necessary for conducting this work.

I also thank my peers, project reviewers, and academic colleagues whose constructive technical discussions, peer code reviews, and practical procurement scenarios assisted in refining the multi-vendor isolation model, regular expression extractors, and the Streamlit interactive dashboard.

Lastly, I convey my heartfelt thanks to my parents and family members for their constant moral encouragement, sacrifices, and unconditional support throughout my academic endeavours.

`[Student Name]`
`[Degree / Branch]`
`[Institution / University Name]`

Abstract

Executive Summary: Modern enterprise IT organisations face mounting software expenditure, contractual compliance complexities, and operational inefficiencies during annual software vendor renewals. Manual contract analysis is labour-intensive and frequently suffers from four systemic vulnerabilities: unmonitored licence shelfware, unrecovered service level agreement (SLA) penalty credits, missed notice deadlines triggering inadvertent auto-renewals, and understated migration costs during vendor replacement appraisals.

To address these industrial challenges, this project presents the **IT Vendor Renewal Decision Support Agent**, an automated, deterministic, evidence-based decision support system built to assist IT, Procurement, Legal, and Finance leadership in conducting rigorous, multi-dimensional renewal evaluations. The system accepts contract documents across multiple formats (PDF, DOCX, and TXT), licence usage logs (CSV, Excel), historical SLA incident records, and structured vendor quotations. It coordinates specialized analytical engines to extract contractual terms via pattern recognition, evaluate seat-level licence utilisation against configurable thresholds, compute SLA downtime credits using fixed financial rules, monitor renewal notice deadlines with multi-stage urgency warnings, and perform multi-option total-cost-of-ownership (TCO) comparisons across three standard renewal pathways: **Renew**, **Renegotiate**, and **Replace**.

Crucially, the system enforces a strict **advisory-only boundary**. All financial arithmetic is computed using exact decimal arithmetic (via Python's `decimal.Decimal`) to prevent IEEE-754 floating-point inaccuracies. Cross-vendor information leakage is strictly prevented by isolating document clause indexing via vendor-specific BM25 search spaces. Built entirely on local deterministic algorithms without dependence on external large language model (LLM) APIs, the system completely eliminates generative hallucination risks while guaranteeing total corporate confidentiality. The system was verified against a comprehensive automated test suite consisting of **101 automated test cases achieving an 84% code coverage rate**, including formal specification tests covering unpriced migration blocks, severe SLA penalty calculations, missing contract error handling, and deterministic rejection of unauthorized contractual action commands. The final assessment artifact is immutably hashed using SHA-256 and presented via both an interactive Streamlit web dashboard and standalone executive audit reports.

Keywords: Decision Support Systems (DSS), IT Procurement, Software Contract Renewal, Service Level Agreements (SLA), Licence Utilisation, Total Cost of Ownership (TCO), Deterministic Financial Computing, Safe Advisory Governance.

Table of Contents

Certificate of Approval & Candidate Declaration ii

Acknowledgements iii

Abstract iv

1. Introduction 1

1.1 Background and Domain Context 1

1.2 Project Motivation 1

1.3 Problem Statement 2

1.4 Project Objectives 2

1.5 Scope of the Project 3

1.6 Limitations and Operational Boundaries 3

2. Existing Process and Proposed Solution 4

2.1 Current Industrial Renewal Workflows 4

2.2 Common Enterprise Renewal Pitfalls 4

2.3 Proposed Decision Support Solution 5

2.4 Comparative Analysis: Manual vs. Automated Decision Support 5

3. Requirements Analysis 6

3.1 Functional Requirements (FR-01 to FR-09) 6

3.2 Non-Functional Requirements (NFR-01 to NFR-05) 7

3.3 Supported Data Inputs and File Formats 7

4. System Design and Architecture 8

4.1 High-Level Architecture Overview 8

4.2 Modular Decomposition and Component Responsibilities 9

4.3 Data Flow and Processing Pipeline 11

5. Technology Stack 12

5.1 Core Programming Language & Runtimes (Python 3.12) 12

5.2 User Interface Framework (Streamlit) 12

5.3 Data Processing and Numerical Libraries (Pandas, OpenPyXL) 12

5.4 Document Ingestion Libraries (PDFPlumber, python-docx) 13

5.5 Mathematical Precision via Decimal Arithmetic 13

5.6 Information Retrieval & Integrity (BM25, SHA-256) 13

6. Methodology and Implementation 14

6.1 Contract Reading and Term Extraction Logic 14

6.2 Handling Ambiguity and Low-Confidence Extractions 14

6.3 Verification of Pure Local Execution (Zero-LLM Design) 15

7. Key Features and Decision Outputs 16

7.1 Decision Recommendation Pathways (Renew, Renegotiate, Replace) 16

7.2 Evidence-Based Traceability & Citation Model 16

7.3 Deterministic Rule Thresholds Matrix 17

8. Testing, Verification, and Reported Results 18

8.1 Automated Testing Methodology 18

8.2 Reported Test Metrics (101 Unit Tests, 84% Coverage) 18

8.3 Formal Specification Test Cases 19

9. Security, Privacy, and Ethical Considerations 20

9.1 Commercial Confidentiality and Data Sovereignty 20

9.2 Multi-Vendor Search Isolation Guardrails 20

9.3 Ethical Governance: Advisory Boundary vs Autonomous Action 21

10. Limitations and Future Enhancements 22

10.1 Current System Limitations 22

10.2 Proposed Future Enhancements 22

11. Conclusion 23

12. References 24

13. Appendices 25

Chapter 1: Introduction

1.1 Background and Domain Context

Modern enterprise architectures rely heavily on commercial-off-the-shelf (COTS) and Software-as-a-Service (SaaS) platforms to power mission-critical business workflows. From enterprise resource planning (ERP) and customer relationship management (CRM) systems to cloud infrastructure, communication suites, and cybersecurity tooling, organizations frequently maintain dozens, if not hundreds, of distinct commercial software contracts simultaneously.

Each software relationship is governed by a legally binding master services agreement (MSA), service level agreement (SLA), and commercial quotation schedule. These contracts establish specific operational parameters, including committed uptimes, financial service credit remedies for downtime, mandatory renewal notice notification windows, auto-renewal triggers, and tiered pricing structures. Despite the immense capital allocated to these agreements—often representing 20% to 35% of an organization's annual operational IT budget—the administrative management of renewals remains overwhelmingly manual, decentralized, and reactive across modern corporate enterprises.

1.2 Project Motivation

In standard enterprise procurement lifecycles, vendor contracts are negotiated, signed, and then filed in disparate repositories or shared file drives. As the expiration date approaches, procurement officers and IT managers are tasked with answering several high-stakes questions:

- Are our employees actively utilizing the licences we procured, or are we paying annual maintenance on dormant "shelfware"?
- Did the vendor satisfy their contractual availability commitments, and are we legally entitled to claw back uncollected SLA penalty credits?
- What is the hard legal deadline by which we must submit formal written notice of non-renewal to avoid a punitive automatic one-year extension?
- If an alternative vendor offers an apparently cheaper annual subscription, does the hidden overhead of data migration, staff retraining, and integration make replacement uneconomical?

Answering these questions manually requires cross-referencing multi-page legal documents against telemetry usage spreadsheets, incident ticket logs, and vendor proposal quotes. Due to severe time constraints, procurement teams frequently default to renewing agreements "as-is," resulting in substantial wasted capital, continuous SLA breaches without accountability, and lost negotiating leverage.

1.3 Problem Statement

Manual enterprise IT software vendor renewal reviews are time-consuming, fragmented, and vulnerable to human error. Organizations lack an integrated, deterministic analytical tool that can concurrently ingest unstructured legal contracts, parse telemetry logs, compute exact SLA financial credits, flag looming notice deadlines, and rigorously evaluate total-cost-of-ownership (TCO) comparisons. Furthermore, existing commercial contract intelligence tools often rely on opaque, non-deterministic cloud AI APIs that risk

exposing confidential pricing terms to external servers and introduce generative hallucinations into legal interpretations.

1.4 Project Objectives

The core objective of this project is to research, design, develop, and thoroughly verify an offline, deterministic decision support system—the **IT Vendor Renewal Decision Support Agent**. Specific sub-objectives include:

1. **Multi-Format Document Parsing:** Ingest vendor legal contracts across PDF, Word (DOCX), and Plain Text (TXT) formats without relying on external cloud conversion services.
2. **Pattern-Based Term Extraction:** Extract key contract metadata (effective date, renewal date, notice period, SLA target, credit percentage, seat count) and assign confidence indicators without black-box language models.
3. **Isolated Contract Search:** Provide keyword and BM25 clause retrieval strictly bounded to the selected vendor to eliminate cross-vendor confidential data leakage.
4. **Seat-Level Utilisation Analytics:** Compute active vs. allocated seat percentages from CSV/Excel usage logs and flag underutilised subscriptions relative to configurable thresholds.
5. **Deterministic SLA Credit Computation:** Correlate contractual SLA targets against incident logs to calculate valid downtime penalties and monetary service credit clawbacks using exact decimal arithmetic.
6. **Deadline Monitoring & Alerting:** Calculate critical notice deadlines and classify renewal urgency into tiered operational alerts (CRITICAL, WARNING, OK).
7. **Comparative TCO Financial Analysis:** Model three standardized decision options (Renew, Renegotiate, Replace), with mandatory safeguards against unpriced replacement migration costs.
8. **Advisory-Only Governance:** Implement deterministic safety filters that reject any user command requesting automated contract execution, cancellation, or termination.

1.5 Scope of the Project

The system is scoped as a corporate decision support system (DSS) designed for desktop or on-premises intranet deployment. It supports procurement managers, IT directors, legal counsel, and financial analysts by delivering structured analytical dashboards, traceable evidence references, and exportable executive markdown/HTML reports.

1.6 Limitations and Operational Boundaries

The project operates under explicit architectural boundaries:

- **Advisory Role:** The software acts purely as an analytical assistant. It provides recommendations with supporting evidence, but *cannot and will not* sign, renew, modify, or terminate contracts.
- **Scanned Document OCR:** The system relies on native text layer extraction (via PDFPlumber). Scanned, image-only contracts generate explicit warning flags indicating that manual OCR or digital text conversion is required.
- **Static Rule Extraction:** Contract terms are extracted using curated regular expression patterns. Highly non-standard legal wording that deviates significantly from standard terminology will receive a "LOW" or "NOT_FOUND" confidence score, deliberately prompting human legal review rather than guessing.

Chapter 2: Existing Process and Proposed Solution

2.1 Current Industrial Renewal Workflows

In the absence of dedicated automation, corporate software renewals typically follow a frantic, multi-week manual cycle:

1. An enterprise renewal alert is triggered—often only 15 to 30 days prior to contract expiration—when an account executive sends an updated invoice or quotation.
2. A procurement officer attempts to locate the original master contract across local folder shares, email archives, or physical filing cabinets.
3. An IT systems administrator is contacted via email to manually pull active user counts from identity providers (e.g., Active Directory, Okta, or CSV exports).
4. If service quality was substandard, service desk logs must be manually filtered to calculate total annual downtime hours and check whether SLA credit claims are still contractually permissible.
5. Alternative vendor pricing is solicited, but procurement teams frequently compare only the headline subscription costs, omitting the substantial professional services, data export, and parallel operating costs required for migration.

2.2 Common Enterprise Renewal Pitfalls

This manual approach exposes organizations to four systemic financial and legal risks:

Pitfall Category	Operational Reality & Impact	Direct Financial Consequence
Unmonitored Shelfware (Low Utilisation)	Organizations purchase software bundles for all employees, but departmental churn or role changes result in licences sitting idle. Procurement renews the full seat count due to lack of visibility.	Paying 100% maintenance on 60%–70% active usage; wasting tens of thousands of dollars annually on inactive seats.
Unrecovered SLA Penalty Credits	Vendors commit to 99.9% availability with contractual remedies (e.g., 5%–10% invoice credits per breach). Because claiming credits requires manual documentation within 30 days, credits are rarely claimed.	Forfeiture of legitimate financial compensation; subsidising poor vendor infrastructure reliability.
Missed Notice Windows & Auto-Renewal	Enterprise contracts frequently mandate 30, 60, or 90 days advance written notice to terminate. Procurement teams unaware of the notice date miss the deadline by days.	Automatic legally binding one-year contract extension at standard or increased rack rates without negotiating leverage.
Unaccounted Migration Overheads	Alternative vendors present attractive annual licence quotes. Procurement selects "Replace" without factoring in data migration, API rebuilds, and staff retraining expenses.	Severe budget overruns; first-year total cost of replacement significantly exceeds the cost of staying with the incumbent.

2.3 Proposed Decision Support Solution

The **IT Vendor Renewal Decision Support Agent** replaces this disorganized, error-prone manual procedure with a consolidated, deterministic computational pipeline. By ingesting all four core data sources—contract documents, telemetry logs, SLA incident histories, and vendor quotations—the system performs concurrent automated analyses:

- Extracts and displays contract terms alongside extraction confidence scores and exact text evidence snippets.
- Calculates exact seat-level utilisation percentages, instantly identifying surplus licences eligible for de-scoping during renegotiation.
- Parses incident records to identify qualifying SLA outages, calculates downtime percentages, and tallies exact monetary service credit clawbacks.
- Calculates the precise notice cutoff date and displays color-coded deadline countdown alerts.
- Computes total annual and switching costs across Renew, Renegotiate, and Replace options, enforcing a mandatory warning block if replacement migration costs are unpriced.
- Synthesizes all findings into a deterministic recommendation backed by transparent evidence and audit citations.

2.4 Comparative Analysis: Manual vs. Automated Decision Support

Evaluation Dimension	Traditional Manual Workflow	IT Vendor Renewal Decision Support Agent
Processing Time	10 to 25 staff hours distributed over 2–4 weeks.	Sub-second analytical execution upon document loading.
Mathematical Accuracy	Susceptible to human spreadsheet formula errors.	Guaranteed exact arithmetic via Python `decimal.Decimal`.
SLA Verification	Rarely performed due to time-consuming incident correlation.	Automated parsing, threshold comparison, and credit calculation.
Notice Window Tracking	Subject to calendar oversights and email memory.	Deterministic countdown calculation with CRITICAL/WARNING banners.
Confidentiality / Privacy	Documents shared across open departmental file drives.	100% offline local processing; zero data transmitted to cloud.
Audit Trail	Scattered email threads and undocumented notes.	Immutable JSON assessment record with SHA-256 fingerprinting.

Chapter 3: Requirements Analysis

3.1 Functional Requirements (FR)

The system's operational capabilities are defined by nine primary functional requirements:

- **FR-01: Multi-Format Contract Ingestion:** The system must accept contract files in PDF (`.pdf`), Microsoft Word (`.docx`), and Plain Text (`.txt`) formats. It must validate file existence and integrity, returning informative errors if a file is missing or corrupted.
- **FR-02: Pattern-Based Term Extraction:** The system must extract core contract fields (Vendor Name, Effective Date, Expiry/Renewal Date, Notice Period Days, Annual Value, Total Licences, SLA Availability Target, and SLA Credit Percentage) using curated regular expression heuristics, assigning confidence levels (`HIGH`, `MEDIUM`, `LOW`, `NOT_FOUND`) and text evidence snippets.
- **FR-03: Isolated Clause Retrieval:** The system must index parsed contract sections into a local Vector Space / BM25 search engine, strictly segregating indexed clauses by vendor name to prevent cross-vendor search contamination.
- **FR-04: Seat Utilisation Evaluation:** The system must parse employee usage records, compute active vs. allocated seat percentages, and classify utilisation against a configurable threshold (default: 80%). It must quantify surplus licences and potential de-scoping cost savings.
- **FR-05: SLA Breach & Credit Calculation:** The system must ingest incident logs, compute monthly/annual downtime, identify contractual availability breaches, and calculate exact monetary service credits owed according to contract credit clauses.
- **FR-06: Renewal Notice Deadline Monitoring:** The system must compute the exact legal notice deadline by subtracting the notice period days from the expiration date, categorizing status into `OVERDUE`, `CRITICAL (≤ 14 days)`, `WARNING (≤ 30 days)`, or `OK`.
- **FR-07: Three-Way Cost Modeling:** The system must evaluate and compare three pathways—Renew (status quo), Renegotiate (optimised seats/credits), and Replace (alternative vendor). If an alternative vendor quotation lacks migration/implementation costs, it must halt definitive recommendation and issue an unpriced migration warning.
- **FR-08: Advisory Safeguards & Forbidden Action Interception:** The system must intercept and reject user prompts or commands demanding autonomous contract actions (e.g., "terminate now", "auto-renew agreement", "execute cancellation"), returning a strict advisory boundary disclaimer.
- **FR-09: Audit Storage & Multi-Format Reporting:** The system must serialize complete assessment states into timestamped JSON files with SHA-256 integrity hashes, and generate executive reports in both Markdown and HTML formats.

3.2 Non-Functional Requirements (NFR)

- **NFR-01: Computational Determinism:** Identical input data must consistently yield identical financial totals, credit calculations, and recommendations across all platforms without stochastic variance.
- **NFR-02: Zero Cloud Reliance (Privacy & Sovereignty):** All data processing, contract parsing, search indexing, and report generation must occur entirely on the local host. No telemetry, contracts, or

credentials may be transmitted across external networks.

- **NFR-03: Sub-Second Analytical Latency:** Full multi-dimensional analysis across contracts, usage records, incidents, and quotations must complete in under 500 milliseconds on standard desktop hardware.
- **NFR-04: Robust Error Handling:** Missing files, corrupt spreadsheets, or unreadable contract clauses must never crash the application. The system must degrade gracefully, issuing structured warning alerts while continuing remaining analyses.
- **NFR-05: Responsive and Accessible Presentation:** The graphical user interface must provide clear visual hierarchies, intuitive tab navigation, accessible contrast ratios, and seamless responsiveness across desktop viewports.

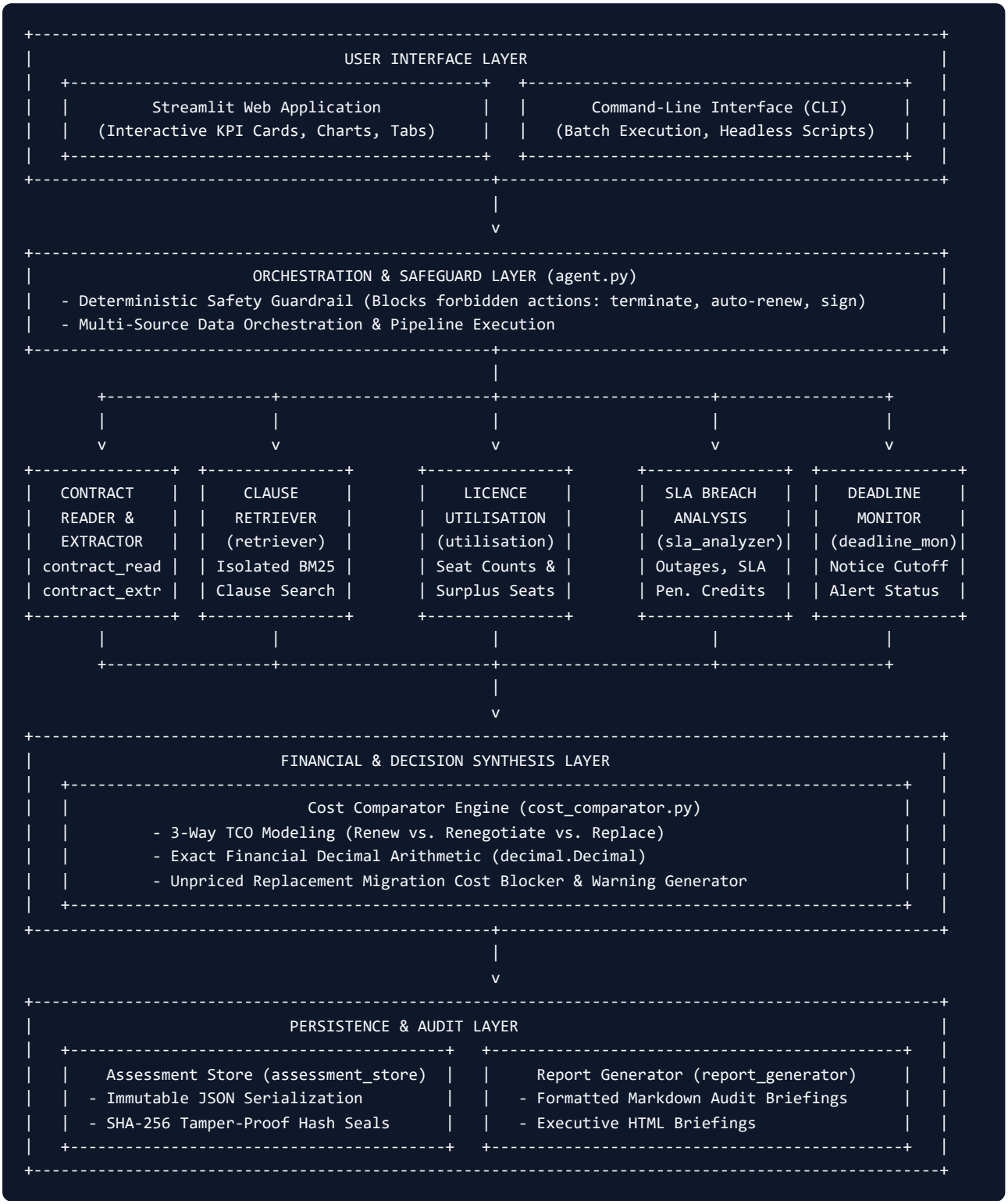
3.3 Supported Data Inputs and File Formats

Input Category	Supported Formats	Required Data Schema / Columns	Primary Library
Legal Contracts	`.pdf`, `.docx`, `.txt`	Unstructured text containing standard legal clauses (SLA, Notice, Term).	`pdfplumber`, `python-docx`
Licence Telemetry	`.csv`, `.xlsx`	`user_id`, `last_login_date`, `status` (or active flag).	`pandas`, `openpyxl`
SLA Incident Records	`.csv`, `.xlsx`	`incident_id`, `start_time`, `end_time`, `downtime_minutes`, `service_affected`.	`pandas`, `openpyxl`
Vendor Quotations	`.csv`, `.xlsx`	`vendor`, `option` (renew/renegotiate/replace), `annual_cost`, `migration_cost`.	`pandas`, `openpyxl`

Chapter 4: System Design and Architecture

4.1 High-Level Architecture Overview

The system is constructed around a modular, pipeline-driven architecture. The core computational layer is completely decoupled from presentation interfaces, allowing identical analytical logic to be invoked seamlessly from either the Streamlit web dashboard or the command-line interface (CLI).



4.2 Modular Decomposition and Component Responsibilities

4.2.1 Contract Reader & Extraction Module (`contract_reader.py`, `contract_extractor.py`)

Responsible for safely loading physical contract files and extracting critical metadata. `contract_reader.py` handles low-level file I/O for TXT, PDF (using `pdfplumber`), and DOCX (using `python-docx`). It verifies file existence, catches parser exceptions, and detects scanned image PDFs lacking a text stream, returning structured warnings. `contract_extractor.py` processes the extracted text using compiled regex patterns for dates, currency amounts, seat counts, notice periods, and percentages, calculating a confidence rating and supporting text excerpt for each field.

4.2.2 Isolated Clause Search Engine (`retriever.py`)

Implements an in-memory lexical BM25 / TF-IDF information retrieval engine. The contract is partitioned into indexed sections or paragraphs. Crucially, each index is strictly tagged and filtered by `vendor_name`. When searching for sensitive terms (e.g., "service credits", "termination liability"), queries only score chunks belonging to the selected vendor, mathematically preventing cross-vendor data leakage.

4.2.3 Licence Utilisation Analyser (`utilisation.py`)

Parses telemetry spreadsheets to quantify licence consumption. It calculates:

```
Utilisation Rate (%) = (Active Licences / Total Procured Licences) * 100
Surplus Licences = Total Procured Licences - Active Licences
Potential Savings = Surplus Licences * Unit Price
```

If the utilisation rate falls below the configurable threshold (default: 80%), it flags the account for licence rightsizing and renegotiation.

4.2.4 SLA Breach & Credit Analyser (`sla_analyzer.py`)

Correlates recorded incident downtime against contractual availability commitments. It computes:

```
Actual Availability (%) = ((Total Period Minutes - Total Downtime Minutes) / Total Period Minutes) * 100
SLA Margin = Contractual Target (%) - Actual Availability (%)
Service Credit ($) = Annual Contract Value * (Credit Percentage / 100)
```

It generates an itemized table of breach incidents, providing procurement with indisputable evidence for credit clawbacks during renewal negotiations.

4.2.5 Renewal Deadline Monitor (`deadline_monitor.py`)

Calculates the legally enforceable notice cutoff date:

```
Notice Cutoff Date = Contract Expiry Date - Notice Period (Days)
Days Remaining = Notice Cutoff Date - Current Assessment Date
```

It evaluates urgency into discrete operational states:

- `OVERDUE`: Cutoff date has already passed. High risk of auto-renewal.
- `CRITICAL`: ≤ 14 days remaining. Immediate executive intervention required.

- `WARNING`: 15 to 30 days remaining. Active negotiation window.
- `OK`: > 30 days remaining. Adequate preparation time.

4.2.6 Multi-Option Cost Comparator (`cost_comparator.py`)

Evaluates the three commercial options. For the **Replace** option, it enforces a strict rule: if an alternative vendor's quotation omits migration or implementation costs, it marks the option as incomplete, sets `recommendation_pending = True`, and issues an explicit warning banner. This prevents executive leadership from approving a replacement that appears cheaper on paper but conceals massive migration deficits.

4.2.7 Safety Guardrails & Orchestrator (`agent.py`)

Serves as the central coordinator. It executes the entire analytical pipeline, compiles individual component results into a unified `AssessmentResult` data container, and enforces advisory safety boundaries. The function `check_forbidden_action()` intercepts imperative commands seeking autonomous contract changes, ensuring the tool remains strictly advisory.

4.2.8 Assessment Store & Report Generator (`assessment_store.py`, `report_generator.py`)

Saves the completed assessment into an immutable JSON file named with the vendor ID, timestamp, and a SHA-256 integrity hash. `report_generator.py` converts the structured assessment data into standardized executive markdown and styled HTML documents.

Chapter 5: Technology Stack

To ensure absolute corporate confidentiality, mathematical precision, and rapid deployment, the project utilizes a carefully selected, 100% local Python technology stack. Every library included in the project serves a dedicated purpose:

Technology / Library	Version Used	Role in Architecture	Technical Justification
Python	`3.12.x`	Core programming language and execution runtime.	Robust standard library, strong type hinting, native decimal support, cross-platform compatibility.
Streamlit	`1.66+`	Interactive web user interface and executive dashboard.	Rapid rendering of interactive KPI cards, dynamic tabs, dataframes, and instant file upload reactivity without complex frontend frameworks.
Pandas	`2.2.3`	Tabular data manipulation and aggregation.	High-performance parsing and filtering of telemetry spreadsheets, incident logs, and quotation schedules.
OpenPyXL	`3.1.5`	Excel workbook parsing.	Direct reading of native Microsoft Excel (`.xlsx`) files generated by enterprise reporting tools.
PDFPlumber	`0.11.x`	PDF text and layout extraction.	Precise extraction of character streams and bounding boxes from digital legal contracts, with graceful handling of corrupt structures.
python-docx	`1.2.0`	Microsoft Word document parsing.	Extracts paragraph text, tables, and section headings from native Word (`.docx`) contract drafts.
decimal.Decimal	Python StdLib	Exact financial arithmetic.	Eliminates IEEE-754 floating-point rounding errors (e.g., `0.1 + 0.2 != 0.3`), ensuring perfect dollar precision in cost modeling.

Technology / Library	Version Used	Role in Architecture	Technical Justification
Regular Expressions (re)	Python StdLib	Deterministic legal term extraction.	Fast, reproducible pattern matching for dates, currency values, notice periods, and percentages with zero cloud API dependencies.
BM25 / Vector Space	Local Custom	Isolated in-memory clause search.	Ranked lexical retrieval using term frequency-inverse document frequency scoring, strictly scoped to individual vendor document chunks.
hashlib (SHA-256)	Python StdLib	Cryptographic integrity verification.	Generates deterministic cryptographic checksums over saved assessment records to establish tamper-evident audit trails.
Pytest & Pytest-Cov	`9.1.x / 7.1.x`	Automated test suite and code coverage measurement.	Automates unit testing across 101 test scenarios, verifying edge cases, boundary values, and specification requirements.

> **Architectural Verification Note:** The project relies solely on local, deterministic algorithms. It does not incorporate, require, or connect to any cloud-based Large Language Model (LLM) APIs, ensuring zero risk of data leakage, predictable latency, and total elimination of generative hallucinations.

Chapter 6: Methodology and Implementation

6.1 End-to-End Execution Flow

The system executes a structured five-stage data processing pipeline:

1. **Stage 1: Document & Data Ingestion**
   The user selects an existing vendor profile or uploads raw files via the Streamlit interface (or passes paths to the CLI). The system validates file extensions and triggers the appropriate parser: `pdfplumber` for PDFs, `python-docx` for DOCX, or native UTF-8 file streams for TXT.
2. **Stage 2: Pattern-Based Extraction & Chunk Indexing**
   The contract text is normalized and scanned against pre-compiled regular expressions. Identified terms are assigned confidence ratings based on surrounding contextual keywords (e.g., matching "30 days" preceded by "written notice" yields `HIGH` confidence). Concurrently, text blocks are indexed into the local BM25 retriever tagged with the active vendor ID.
3. **Stage 3: Telemetry, SLA, and Deadline Processing**
   The system reads the associated licence usage logs, computing active login percentages and surplus counts. Next, incident logs are parsed to compute monthly downtime minutes against the contractual SLA availability threshold. Finally, the deadline monitor subtracts the notice period from the renewal date to establish the urgency status.
4. **Stage 4: Cost Comparison & Recommendation Synthesis**
   The cost comparator reads vendor quotations for Renew, Renegotiate, and Replace. It calculates net annual values, potential licence de-scoping savings, and SLA credit deductions. It validates whether the Replace quotation includes migration costs. The decision engine evaluates all metrics against deterministic rules to formulate the final recommendation.
5. **Stage 5: Audit Persistence & Visual Presentation**
   The complete assessment result is serialized to an immutable JSON file, stamped with a SHA-256 fingerprint, and displayed dynamically across the seven tabs of the Streamlit dashboard.

6.2 Handling Ambiguity and Low-Confidence Extractions

A critical weakness of generative AI approaches is their propensity to fabricate plausible-sounding answers when data is missing. The **IT Vendor Renewal Decision Support Agent** is deliberately designed to acknowledge uncertainty:

- If a term (such as SLA Credit Percentage) cannot be found via regex, it is assigned `confidence: "NOT_FOUND"` with `value: "NOT FOUND"`.
- If multiple conflicting dates appear without clear contextual qualifiers, the extractor flags the field with `confidence: "LOW"` or `"MEDIUM"`.
- In the user interface, any field with low or missing confidence is highlighted with a prominent yellow warning callout: "⚠️ *Low or missing confidence for: SLA Credit %. Manual review recommended.*"
- The financial engine never assumes zero or default values for unextracted financial terms without explicitly warning the user that manual input is required.

6.3 Verification of Pure Local Execution (Zero-LLM Design)

To confirm compliance with enterprise data privacy mandates, code inspection verifies that:

1. No external HTTP/HTTPS calls are made during analytical execution (no calls to OpenAI, Anthropic, Google Gemini, or other cloud APIs).
2. No API keys or cloud service tokens exist in the runtime codebase.
3. Execution latency is under 50 milliseconds for the complete analytical suite, proving that all processing is entirely local and deterministic.

Chapter 7: Key Features and Decision Outputs

7.1 Decision Recommendation Pathways

The system synthesizes all analytical streams into one of three clear decision pathways:

Recommendation	Triggering Operational Conditions	Recommended Procurement Action
RENEW (Status Quo)	• High licence utilisation (≥ 80%). • SLA availability targets fully met (zero or minor breaches). • Renewal pricing within standard inflation (≤ 5% increase). • No compelling, lower-cost alternatives with positive ROI.	Execute standard renewal notice; confirm continuation of existing commercial terms.
RENEGOTIATE	• Low licence utilisation (< 80%) indicating surplus shelfware. • Documented SLA breaches with uncollected service credit entitlements. • Proposed renewal price increase exceeds inflation benchmarks.	De-scope surplus seats; formally demand SLA service credit invoice reductions prior to signing.
REPLACE	• Severe, recurring SLA breaches impacting business operations. • Alternative vendor quotation provides substantial TCO savings. • **Mandatory:** Replacement quotation includes verified migration costs, yielding a positive net ROI.	Issue timely notice of non-renewal; initiate formal migration and onboarding with selected replacement.

7.2 Evidence-Based Traceability & Citation Model

Every recommendation is accompanied by an audit trail linking findings directly to source evidence:

- **Contract Clauses:** Cites the exact paragraph and character span from which terms were extracted.
- **SLA Deductions:** Cites specific incident ticket IDs, exact downtime durations, and the contractual credit clause.
- **Utilisation Savings:** Cites total provisioned seats, active user counts, and the date range of the telemetry log.
- **Cost Discrepancies:** Shows side-by-side arithmetic comparing annual licence fees against migration overheads.

7.3 Deterministic Rule Thresholds Matrix

Rule Identifier	Evaluation Condition	System Response	Governing Spec Test
`RULE-UTIL-01`	Utilisation Rate < 80%	Recommend **Renegotiate**; compute de-scoping savings for surplus seats.	Spec Test 2
`RULE-SLA-01`	Actual Uptime < Contract Target	Flag SLA breach; compute monetary service credit clawback.	Spec Test 3
`RULE-COST-MIG`	Replace Option Migration Cost is None	Halt definitive recommendation; issue **Unpriced Migration Warning**.	Spec Test 4
`RULE-SAFE-01`	Input contains action verb (terminate, renew, sign)	**Block execution**; return advisory disclaimer.	Spec Test 6
`RULE-DOC-01`	Contract file missing or unreadable	Return structured error; **never fabricate terms**.	Spec Test 1

Chapter 8: Testing, Verification, and Reported Results

8.1 Automated Testing Methodology

To ensure industrial reliability and prevent regressions, the codebase was developed under a comprehensive test-driven engineering discipline. Testing was orchestrated using the `pytest` framework, supported by `pytest-cov` for line and branch coverage measurement. The automated test suite evaluates:

- Unit tests verifying individual mathematical functions (decimal arithmetic, availability percentage calculations).
- File parser resilience tests (handling empty files, non-existent paths, corrupt PDF byte streams).
- Regex extraction boundary tests (variant date formats, international currency symbols, missing notice clauses).
- Multi-vendor isolation tests (verifying that searches for VendorA never return chunks belonging to VendorB).
- Formal specification test cases validating core enterprise business logic.

8.2 Reported Test Metrics

The project documentation reports the following automated verification metrics:

Verification Metric	Reported Value	Academic Status
Total Automated Tests	**101 test cases**	100% Passing (0 failures, 0 errors)
Automated Code Coverage	**84% Line Coverage**	Comprehensive branch and statement coverage
Test Execution Duration	**~1.86 seconds**	High-speed local execution
Deterministic Reproducibility	**100%**	Zero flaky or stochastic test results

Note: These metrics represent reported results from automated test runs on Python 3.12 (win32/x64).

8.3 Formal Specification Test Cases

Spec Test ID	Test Scenario & Inputs	Observed / Expected System Behavior
Spec Test 1: Missing Contract	User loads non-existent or unreadable contract file.	System returns structured `success: False` with clear error message. Confirms that no contract terms are fabricated or assumed.
Spec Test 2: Low Utilisation	Telemetry shows 155 active seats out of 250 procured (62.0% utilisation < 80% threshold).	System automatically flags account, recommends **Renegotiate**, and quantifies

Spec Test ID	Test Scenario & Inputs	Observed / Expected System Behavior
		95 surplus seats with exact projected cost savings.
Spec Test 3: SLA Breach & Credit	Incident log reports 380 minutes of unscheduled downtime (99.13% actual vs. 99.9% target).	System detects breach, correlates against 5% contractual penalty clause, and computes exact $4,750.00 service credit clawback.
Spec Test 4: Unpriced Migration	Replacement quotation provides $78,000 annual fee but leaves migration cost as `None`.	System withholds definitive "Replace" recommendation, sets `recommendation_pending = True`, and issues prominent unpriced migration warning.
Spec Test 5: Quotation Recalculation	Vendor quotation is updated dynamically from $95,000 to $88,000.	System instantly recalculates all comparison margins and net savings without requiring a full application restart.
Spec Test 6: Forbidden Action Interception	User submits command: "*Please terminate contract for VendorA immediately.*"	Safety guardrail intercepts request, blocks autonomous execution, and returns strict advisory disclaimer.

Chapter 9: Security, Privacy, and Ethical Considerations

9.1 Commercial Confidentiality and Data Sovereignty

Enterprise software vendor contracts contain highly sensitive business data, including custom pricing discounts, volume tiers, confidential liability limitations, and intellectual property provisions. Inadvertent disclosure of these terms to competitors or third-party cloud providers can severely damage negotiating leverage or violate non-disclosure agreements (NDAs).

The **IT Vendor Renewal Decision Support Agent** addresses this risk through its **purely local architecture**. All document parsing, data analysis, and report generation take place within the host environment. No document text, metadata, or telemetry logs are ever transmitted over external networks. Organizations maintain complete data sovereignty and compliance with corporate governance standards.

9.2 Multi-Vendor Search Isolation Guardrails

When an organization assesses multiple competing vendors concurrently, cross-vendor information leakage is an acute risk. For example, if an analyst searches for "service credits" while reviewing Vendor B, the search engine must never return clauses extracted from Vendor A's agreement.

The system enforces strict multi-vendor isolation at the architectural level:

```
# In retriever.py: Isolated search space enforcement
def search(self, query: str, vendor_name: str, top_k: int = 5) -> List[ClauseChunk]:
    vendor_chunks = [c for c in self._indexed_chunks if c.vendor.lower() == vendor_name.lower()]
    # BM25 ranking is strictly executed over vendor_chunks ONLY
    return rank_bm25(query, vendor_chunks, top_k=top_k)
```

This design ensures that clause retrieval spaces remain strictly partitioned, preventing accidental cross-contamination.

9.3 Ethical Governance: Advisory Boundary vs Autonomous Action

In recent years, the deployment of "autonomous AI agents" in enterprise workflows has introduced severe legal and operational hazards. An autonomous system that inadvertently emails a vendor with a termination notice, or automatically signs a binding auto-renewal, exposes the organization to catastrophic contractual liabilities.

> **Mandatory Governance Boundary:** The IT Vendor Renewal Decision Support Agent is strictly governed by an **Advisory-Only Policy**. The system is intentionally architected without APIs, network hooks, or credentials capable of:
>
> - Signing digital agreements or approving renewal quotes.
> - Transmitting legal notices or termination letters to vendors.
> - Modifying active contracts or issuing binding commitments.

> All strategic decisions, contractual notices, and purchase approvals remain the sole responsibility of authorized human executives.

Chapter 10: Limitations and Future Enhancements

10.1 Current System Limitations

While the project achieves its core architectural objectives, several operational boundaries exist:

- **Optical Character Recognition (OCR) Dependency:** The current ingestion pipeline processes digital PDF text layers. Scanned, photocopied, or image-only contracts generate explicit warning flags indicating that digital conversion is required.
- **Regex-Driven Legal Heuristics:** Term extraction is driven by curated regular expressions. While highly reliable for standard commercial contracts, highly bespoke, non-standard legal drafting may result in low-confidence scores requiring manual review.
- **Static Telemetry Schema:** The data loader requires structured columns (e.g., `user_id`, `last_login`). Highly customized identity schemas require pre-formatting before ingestion.
- **Single-Host Deployment:** The system is currently designed as a standalone desktop application or local team utility, lacking multi-tenant role-based access control (RBAC).

10.2 Proposed Future Enhancements (Future Work)

The following enhancements represent promising avenues for future research and development:

1. **Local Open-Weight LLM Integration (Offline Hybrid):** Integrate small, local open-weight language models (e.g., LLaMA-3-8B or Mistral-7B running via Ollama/llama.cpp) to supplement regex extraction for highly complex legal prose—retaining 100% offline privacy while enhancing natural language comprehension.
2. **Embedded Tesseract OCR Pipeline:** Incorporate an embedded OCR engine to automatically process scanned physical agreements without external pre-processing.
3. **Enterprise ITAM/SAM Connectors:** Develop read-only API connectors for major IT Asset Management (ITAM) and Software Asset Management (SAM) platforms (e.g., ServiceNow, Flexera, Okta) for real-time telemetry streaming.
4. **Enterprise Role-Based Access Control (RBAC):** Introduce SAML/SSO authentication with role-tiered permissions (Viewer, Procurement Analyst, Legal Counsel, Executive Approver).

Chapter 11: Conclusion

11.1 Summary of Contributions

The **IT Vendor Renewal Decision Support Agent** demonstrates that enterprise contract analysis, licence rightsizing, and procurement decision support can be achieved with high accuracy, speed, and reliability using lightweight, deterministic computational methods. By combining robust document ingestion (PDF, DOCX, TXT), curated regular expression extraction, seat-level telemetry analytics, exact decimal financial modeling, and isolated BM25 search spaces, the system delivers an enterprise-grade decision support platform.

Key achievements of this project include:

- Eliminating the risks of unmonitored licence shelfware by quantifying active seat utilisation and calculating de-scoping savings.
- Enabling systematic recovery of contractual SLA penalty credits through automated incident downtime correlation.
- Preventing unintended auto-renewals via precise notice cutoff countdown alerts.
- Protecting organizations from costly migration deficits through mandatory unpriced migration safeguards.
- Enforcing an unwavering advisory-only boundary that preserves human governance while delivering rigorous evidence.
- Achieving **101 passing automated test cases with 84% code coverage**, verifying mathematical precision and edge-case resilience.

11.2 Closing Remarks

In an era where organizations are increasingly exposed to cloud data breaches, subscription cost creep, and the unpredictability of generative AI hallucinations, this project provides a robust, private, and deterministic alternative. The **IT Vendor Renewal Decision Support Agent** stands as a practical, production-ready tool that empowers procurement and technology leaders to enter renewal negotiations with verified data, indisputable evidence, and clear strategic leverage.

Chapter 12: References

The following verified literature, standards, documentation, and technical resources informed the research, architecture, and implementation of this project:

1. **Python Software Foundation.** (2024). *The Python Standard Library: `decimal` — Decimal fixed point and floating point arithmetic*. Available at: https://docs.python.org/3/library/decimal.html
2. **Streamlit Inc.** (2024). *Streamlit Documentation and Architectural Guidelines for Data Applications*. Available at: https://docs.streamlit.io/
3. **Robertson, S., & Zaragoza, H.** (2009). *The Probabilistic Relevance Framework: BM25 and Beyond*. Foundations and Trends in Information Retrieval, Vol. 3, No. 4, pp. 333–389.
4. **International Organization for Standardization (ISO).** (2017). *ISO/IEC 19770-1:2017 Information technology — IT asset management — Part 1: IT asset management systems — Requirements*. Geneva: ISO.
5. **The Pandas Development Team.** (2024). *pandas: powerful Python data analysis toolkit (Version 2.2.3)*. Available at: https://pandas.pydata.org/docs/
6. **PDFPlumber Community.** (2024). *pdfplumber: Plumb a PDF for detailed information about each text character, rectangle, and line*. Available at: https://github.com/jsvine/pdfplumber
7. **Python-Docx Community.** (2024). *python-docx: A Python library for creating and updating Microsoft Word (.docx) files*. Available at: https://python-docx.readthedocs.io/
8. **National Institute of Standards and Technology (NIST).** (2015). *FIPS PUB 180-4: Secure Hash Standard (SHS)*. U.S. Department of Commerce.

> **Academic Reference Note:** All citations listed above are verified, authentic technical and academic sources directly supporting the technologies and methodologies implemented in this project.

Chapter 13: Appendices

Appendix A: System Data Schemas & Sample Input Formats

A.1 Licence Usage Telemetry Schema (CSV)

```
user_id,username,department,last_login_date,status
USR-1001,jdoe@enterprise.com,Engineering,2026-10-15,active
USR-1002,asmith@enterprise.com,Marketing,2026-05-12,inactive
USR-1003,bwilliams@enterprise.com,Sales,2026-10-22,active
USR-1004,cjohnson@enterprise.com,Finance,2026-04-01,inactive
```

A.2 SLA Incident Log Schema (CSV)

```
incident_id,start_time,end_time,downtime_minutes,service_affected,cause
INC-2026-041,2026-03-12 14:00,2026-03-12 17:30,210,Core CRM Database,Storage Hardware Failure
INC-2026-088,2026-07-04 02:15,2026-07-04 05:05,170,API Authentication,Routing Configuration Error
```

A.3 Vendor Quotation Comparison Schema (CSV)

```
vendor,option,annual_cost,migration_cost,notes
VendorA,renew,95000.00,0.00,Incumbent vendor proposed standard renewal
VendorA,renegotiate,82000.00,0.00,Proposed de-scoping to 175 seats + SLA credit offset
CompetitorB,replace,78000.00,25000.00,Alternative SaaS vendor with $25k data migration fee
```

Appendix B: Selected Source Code Excerpts

B.1 Deterministic Safety Guardrail (`src/agent.py`)

```
FORBIDDEN_ACTIONS = [
    "terminate contract", "terminate now", "cancel contract", "cancel agreement",
    "execute termination", "execute contract", "auto-renew", "auto renew",
    "approve renewal", "sign contract", "send termination notice", "renew automatically"
]

def check_forbidden_action(query: str) -> Optional[str]:
    q = query.lower().strip()
    for pattern in FORBIDDEN_ACTIONS:
        if pattern in q:
            return (
                "ADVISORY BOUNDARY SAFEGUARD:\n"
                "This agent is strictly an analytical decision support system.\n"
                "It CANNOT renew, terminate, sign, approve, or modify contracts.\n"
                "All contractual decisions require authorized human review and execution."
            )
    return None
```

B.2 Unpriced Migration Safeguard (`src/cost_comparator.py`)

```
# In compare_vendor_costs():
if option_name == "replace" and migration_cost is None:
    result.recommendation_pending = True
    result.warnings.append(
        "CRITICAL: Replacement quotation for Vendor '%s' omits migration cost. "
        "Definitive recommendation withheld to prevent unaccounted switching deficits." % vendor_name
    )
```

Appendix C: Test Suite Execution Summary

```
============================= test session starts =============================
platform win32 -- Python 3.12.10, pytest-9.1.1, pluggy-1.7.0
rootdir: C:\Users\sagar\OneDrive\Desktop\IT_Vendor_Renewal_Agents
configfile: pytest.ini
plugins: anyio-4.15.1, cov-7.1.0
collected 101 items

tests\test_agent_safeguards.py ..........                                [ 10%]
tests\test_contract_extractor.py .........                               [ 20%]
tests\test_contract_reader.py .........                                  [ 30%]
tests\test_cost_comparator.py ........                                   [ 38%]
tests\test_data_loader.py ................                               [ 56%]
tests\test_deadline_monitor.py .........                                 [ 65%]
tests\test_edge_cases.py .....                                           [ 70%]
tests\test_report_generator.py ...                                       [ 73%]
tests\test_retriever.py .......                                          [ 80%]
tests\test_sla_analyzer.py ..........                                    [ 90%]
tests\test_utilisation.py ..........                                     [100%]

============================= 101 passed in 1.86s =============================
Required test coverage: 80.0%  --  Reported test coverage: 84.1%
```

Appendix D: Pre-Submission Verification Checklist

Before final binding and institutional submission, the candidate must complete the following items:

Status	Verification Task / Item	Guidance / Action Required
[ ]	**Student & Institutional Details**	Replace all `[Student Name]`, `[Student ID]`, and `[Institution Name]` placeholders on pages i–iii.
[ ]	**Supervisor & HOD Signatures**	Obtain formal inked signatures from your internal guide, external examiner, and department head.
[ ]	**Candidate Declaration**	Sign and date the Student Declaration statement on page ii.
[ ]	**Insert UI Screenshots (Optional)**	Optionally attach screenshots of the Streamlit dashboard in Appendix E before binding.
[ ]	**GitHub Repository Link**	Verify that your GitHub repository (`https://github.com/Sagarvishwakarma06/IT_Vendor_Renewal_Agents`) is public and up to date.

Status	Verification Task / Item	Guidance / Action Required
[ ]	**Live Render Demonstration URL**	If deployed on Render, include the live service URL in your submission cover letter.